Foundations of Computer Science (26CSE0103)

Topic: Software and Programming Language Part-2

Prepared By: Dr. Tajinder Kaur

Chitkara University Institute of Engineering and Technology,
Chitkara University, Punjab

Contents

CHITKARA UNIVERSITY

- Introduction to Programming Languages
- Evolution of Programming Languages
- Machine Language (1GL)
- Assembly Language (2GL)
- Fourth Generation Languages (4GL)
- Fifth Generation Languages (5GL)
- Comparison of Programming Languages
- Language Translators
- Applications

Introduction to Programming Languages

CHITKARA UNIVERSITY

Programming Languages

Programming languages are the medium through which humans communicate with computers.

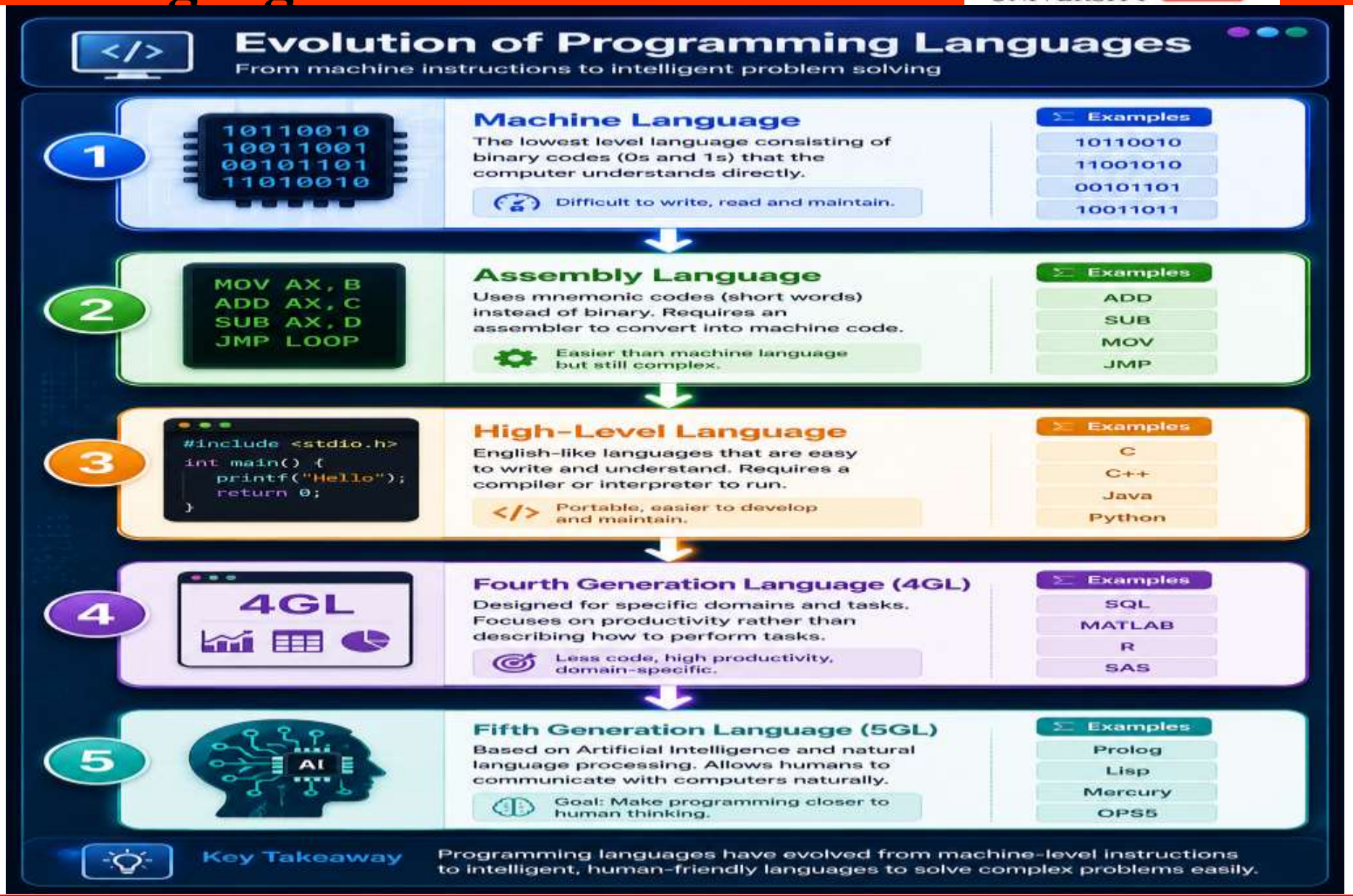
A programming language provides instructions that tell a computer what task to perform.

Objectives

- Understand the evolution of programming languages.
- Study different generations of programming languages.
- Learn about language translators.
- Compare assembler, compiler, and interpreter.

Evolution of Programming Languages

CHITKARA UNIVERSITY



Machine Language (1GL)

CHITKARA UNIVERSITY

Definition

- Machine language consists of binary digits (0 and 1).
- Example:
- 10110010
- 11001100
- 01010101
- The CPU directly understands machine language.
- **Characteristics**
- Written in binary.
- Fast execution.
- Hardware dependent.
- Difficult to write.
- Difficult to debug.
- Time-consuming.

Advantage and Disadvantage of 1GL

CHITKARA UNIVERSITY

- **Advantages**
- No translation required.
- Maximum execution speed.
- **Disadvantages**
- Very difficult to understand.
- Error-prone.
- Poor portability.

Assembly Language (2GL)

- Assembly language uses mnemonics instead of binary numbers.
- Example

```
MOV AX,10
ADD AX,20
SUB AX,5
```

- Assembly language requires an **Assembler** for translation.

Features

- Uses symbolic names.
- Easier than machine language.
- Hardware dependent.
- Faster than high-level languages.

Advantages

- Easier debugging.
- Efficient memory utilization.
- Better readability.
- **Disadvantages**
- Machine dependent.
- Complex for large programs.

High-Level Languages (3GL)

CHITKARA UNIVERSITY

Definition

High-level languages resemble English.

Examples

- C ,C++, Java, Python, C#, Pascal

Features

- User friendly
- Portable
- Easy debugging
- Easy maintenance
- Machine independent

Fourth Generation Languages (4GL)

- Designed for database applications and rapid application development.

Examples

- SQL, MATLAB, Oracle Reports, SAS
- **Features**
- Very little coding
- High productivity
- Faster development
- Used for database management

Applications

- Data analysis
- Report generation
- Business applications

*__SAS__ stands for **Statistical Analysis System**. It is a software suite and **4GL programming language** used for **data analysis, reporting, and statistics**.

Fifth Generation Languages (5GL)

CHITKARA UNIVERSITY

- Designed for Artificial Intelligence.

Examples

- Prolog
- Mercury
- OPS5

Applications

- Robotics planning
- Expert Systems
- AI
- Machine Learning
- Natural Language Processing

3GL: "Do step 1, then step 2, then step 3." (how)

4GL: "Give me the average salary by department."

(what)

5GL: "Here are the rules. Solve this problem."

(solve)

Comparison of Programming Languages

CHITKARA UNIVERSITY

Feature	Machine	Assembly	High-Level
Language	Binary	Mnemonics	English-like
Speed	Very High	High	Moderate
Readability	Very Poor	Good	Excellent
Portability	No	No	Yes
Development Time	High	Medium	Low

Translator

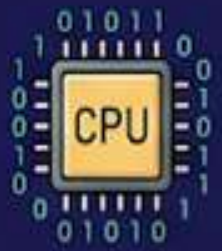
CHITKARA UNIVERSITY

WHAT IS A PROGRAMMING LANGUAGE TRANSLATOR?

Programming translator - software that converts source code to machine language



High-Level Code → TRANSLATOR (Software Tool) → Low-Level Machine Code



TYPES OF PROGRAMMING TRANSLATORS

COMPILER



- Scans all code
- Generates executable (.exe)
- Runs



- Fast execution, slower compilation, errors reported at the end

Example: C, C++

INTERPRETER



- Translates line 1
- Executes line 1
- Translates line 2
- Executes



- Faster development, slower execution, errors reported immediately

Example: Python, JavaScript

ASSEMBLER



- Converts Assembly Language (Mnemonic code like MOV, ADD) directly to Machine Code
- Fast, low-level, machine-specific



x86 Assembly

HYBRID TRANSLATOR (JIT)



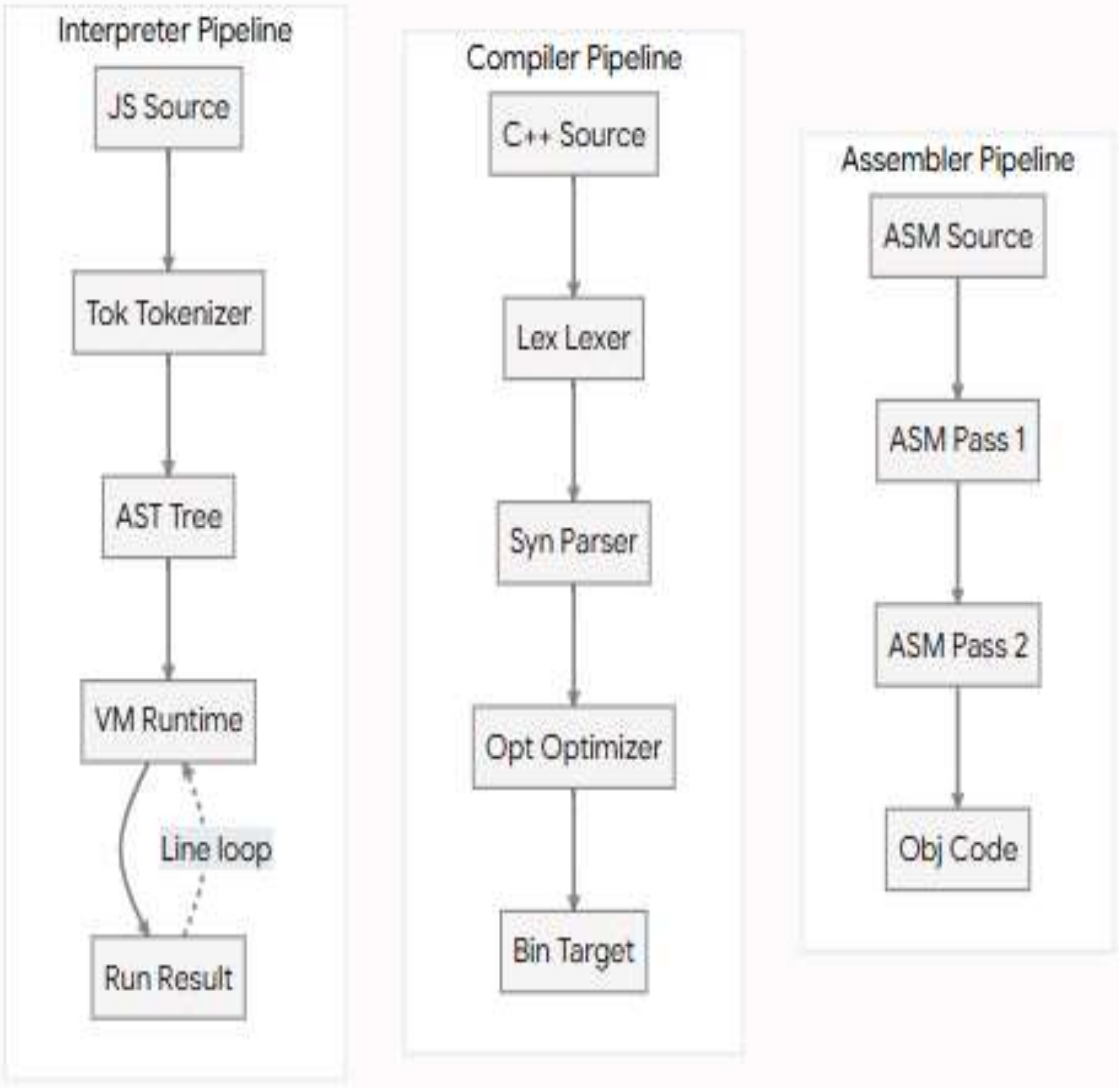
- Combines Compilation & Interpretation
- Compiles code to Bytecode, then interprets or compiles Bytecode during runtime
- Faster start-up, better performance than pure interpretation

Example: Java, C#

- **Language Translator architecture visualizer**

CHITKARA UNIVERSITY

An interactive ***language translator architecture visualizer*** allows you to trace source code through each translation path—**Assembler**, **Compiler**, and **Interpreter**—down to machine execution.



Interpreter Pipeline

1. JS Source — JavaScript Source Code

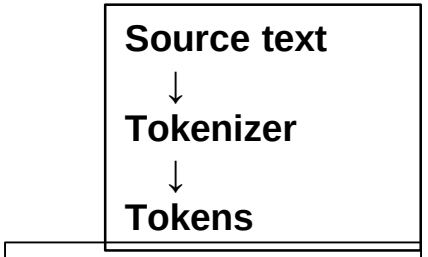
This is the program written by the programmer.

```
let x = 10;
let y = 20;
console.log(x + y);
```

2. Tok Tokenizer

Tokenizer = breaks source code into tokens.

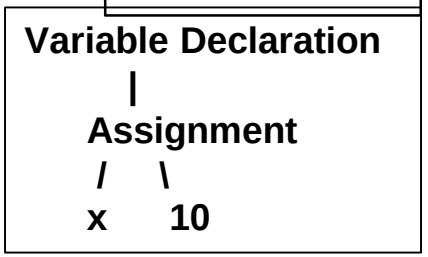
A token is a meaningful piece of the program, such as: keyword, identifier



3. AST Tree — Abstract Syntax Tree

AST = Abstract Syntax Tree.

The parser uses the tokens to construct a tree representing the program's structure.



4. VM Runtime — Virtual Machine Runtime

VM = Virtual Machine.

The VM executes the program represented by the AST or by an intermediate representation derived from it.

- **Create x**
- **Store 10**
- **Create y**
- **Store 20**
- **Calculate x + y**
- **Call console.log**
- **Produce 30**

5. Run Result

This is the **observable result of executing the program**.

Compiler Pipeline

CHITKARA UNIVERSITY

C++ Source → Lex Lexer → Syn Parser → Opt Optimizer → Bin Target

```
int add(int a, int b)
{
    return a + b;
}
```

1. C++ Source
This is the program written in C++.

2. Lex Lexer
Lexer = lexical analyzer.
The lexer reads the source code and converts it into **tokens**.

```
int x = 10;
int  → keyword/type
x    → identifier
=    → assignment operator
10   → integer literal
;    → punctuation
```

3. Syn Parser
Parser = analyzes the grammatical structure of the tokens.

4. Opt Optimizer
Optimizer = improves the program while preserving its intended behavior.

```
int x = 10 * 20;
An optimizer can determine that:
10 × 20 = 200
```

5. Bin Target
Bin = binary/target output.
This represents the compiler's target representation i.e. Machine code

3. Assembler Pipeline

ASM Source → ASM Pass 1 → ASM Pass 2 → Obj Code

1. ASM Source
ASM means **assembly language**.

```
MOV R1, 10
ADD R1, R2
```

2. ASM Pass 1
A traditional assembler may use **two passes**.
During **Pass 1**, the assembler reads the assembly source and typically determines things such as:

- instruction locations
- sizes of instructions/data

3. ASM Pass 2
During **Pass 2**, the assembler uses the information collected during Pass 1 to generate the actual machine/object representation.

4. Obj Code — Object Code
Object code = machine-oriented code produced by the assembler/compiler, usually stored in an object file.
It is much closer to what the CPU can execute than assembly source.

Address	Instruction	Hex Code	Bytes	Comment
2000	MVI A,05H	3E 05	2	A = 05
2002	MVI B,03H	06 03	2	B = 03
2004	ADD B	80	1	A = A + B = 08
2005	HLT	76	1	Stop

Program: add two numbers using MVI

MVI puts a number **directly into a register**, so no memory address is needed.

Total size = 2 + 2 + 1 + 1 = 6 bytes

How the addresses are calculated

2000 + 2 bytes (MVI A) = 2002

2002 + 2 bytes (MVI B) = 2004

2004 + 1 byte (ADD B) = 2005

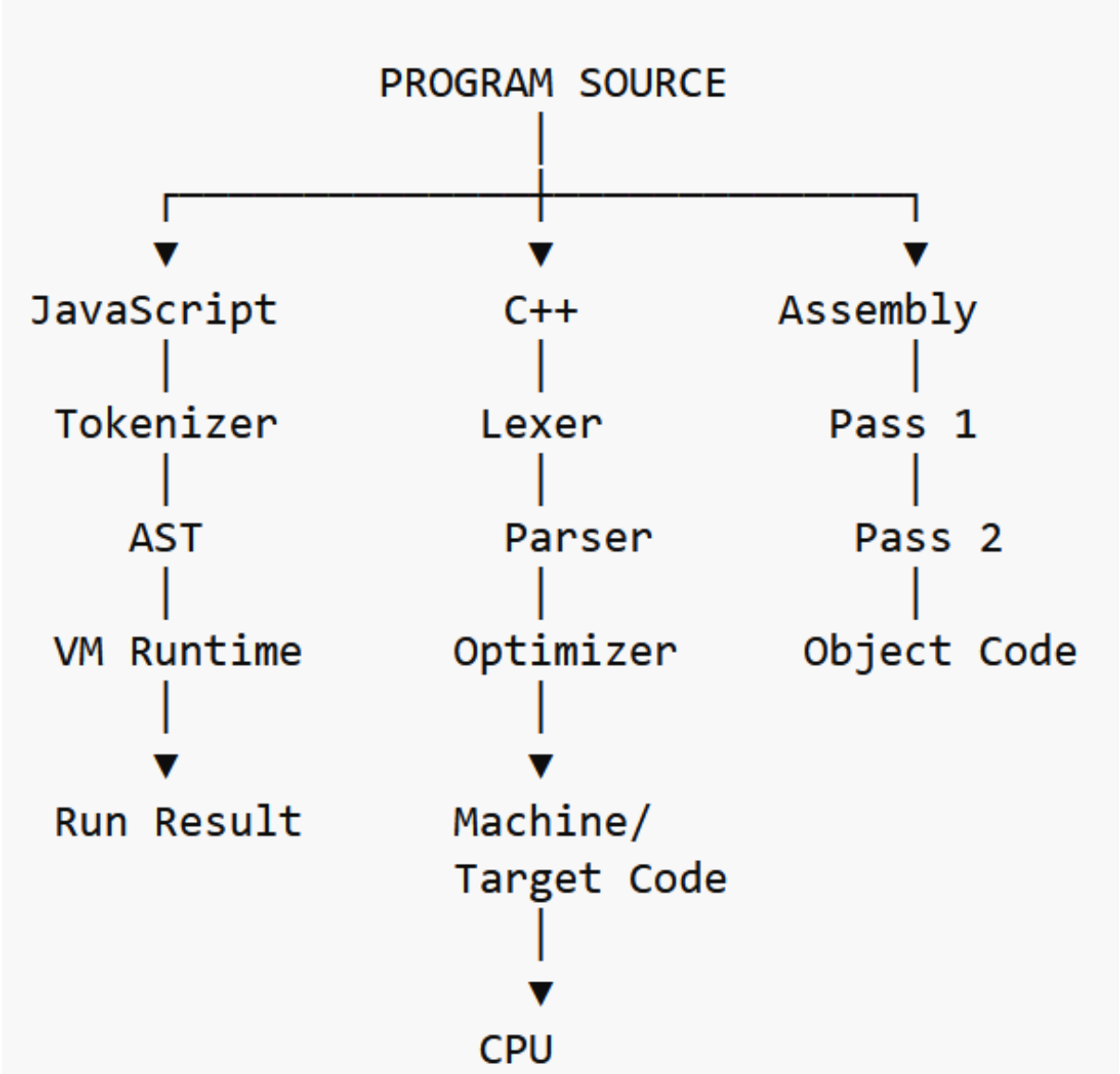
2005 + 1 byte (HLT) = 2006 (end)

Machine code

3E 05 06 03 80 76

How it sits in memory

Memory Address	Content (Hex)	Part of
2000	3E	MVI A (opcode)
2001	05	data for A
2002	06	MVI B (opcode)
2003	03	data for B
2004	80	ADD B
2005	76	HLT


PROGRAM SOURCE
JavaScript
C++
Assembly
Tokenizer
Lexer
Pass 1
AST
Parser
Pass 2
VM Runtime
Optimizer
Object Code
Run Result
Machine/
Target Code
CPU

Assembler

CHITKARA UNIVERSITY

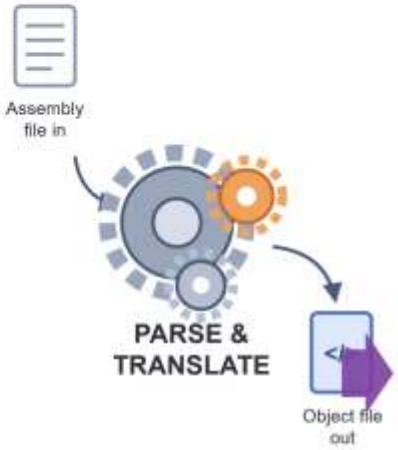
ASSEMBLY PROGRAM

```
1 MOV AX, 10
2 ADD AX, BX
3 ADD AX, IN
4
5 INT 21H
6 HLT
```

- Source Code Input
- Mnemonic Language
- Human-Readable Format

Language: ASM

ASSEMBLER



- Translates Mnemonics
- Generates Object Code
- Reports Errors

Progress:

MACHINE CODE

```
00000000: 0000 0000 0010 0111
00000010: 0101 0011 0110 0110
00000020: 0101 0011 0110 0001
00000030: 0101 0001 0011 0100
00000040: 0000 0111 0000 0010
00000050: 0101 0011 0100 0110
00000060: 0000 0111 0100 0100
00000070: 0101 0011 0101 0101
00000080: 0111 0010 0100 0
```

- Low-Level Output
- Directly Executable (Binary)
- Hardware-Specific

Example:

```
00000000: 000 data
00000010: 000 0011 ...
00000020: 000 0011 ...
00000030: 000 0110 ...
```

EXECUTION



- Program Running

Status:

Program Running Successfully



Advantage and disadvantage of Assembler

CHITKARA UNIVERSITY

Advantages

- Fast execution
- Efficient code
- Direct hardware control

Disadvantages

- Machine dependent
- Difficult programming

Compiler

CHITKARA UNIVERSITY

Definition

- A compiler translates the **entire program** before execution.



Advantage and Disadvantage of Compiler

CHITKARA UNIVERSITY

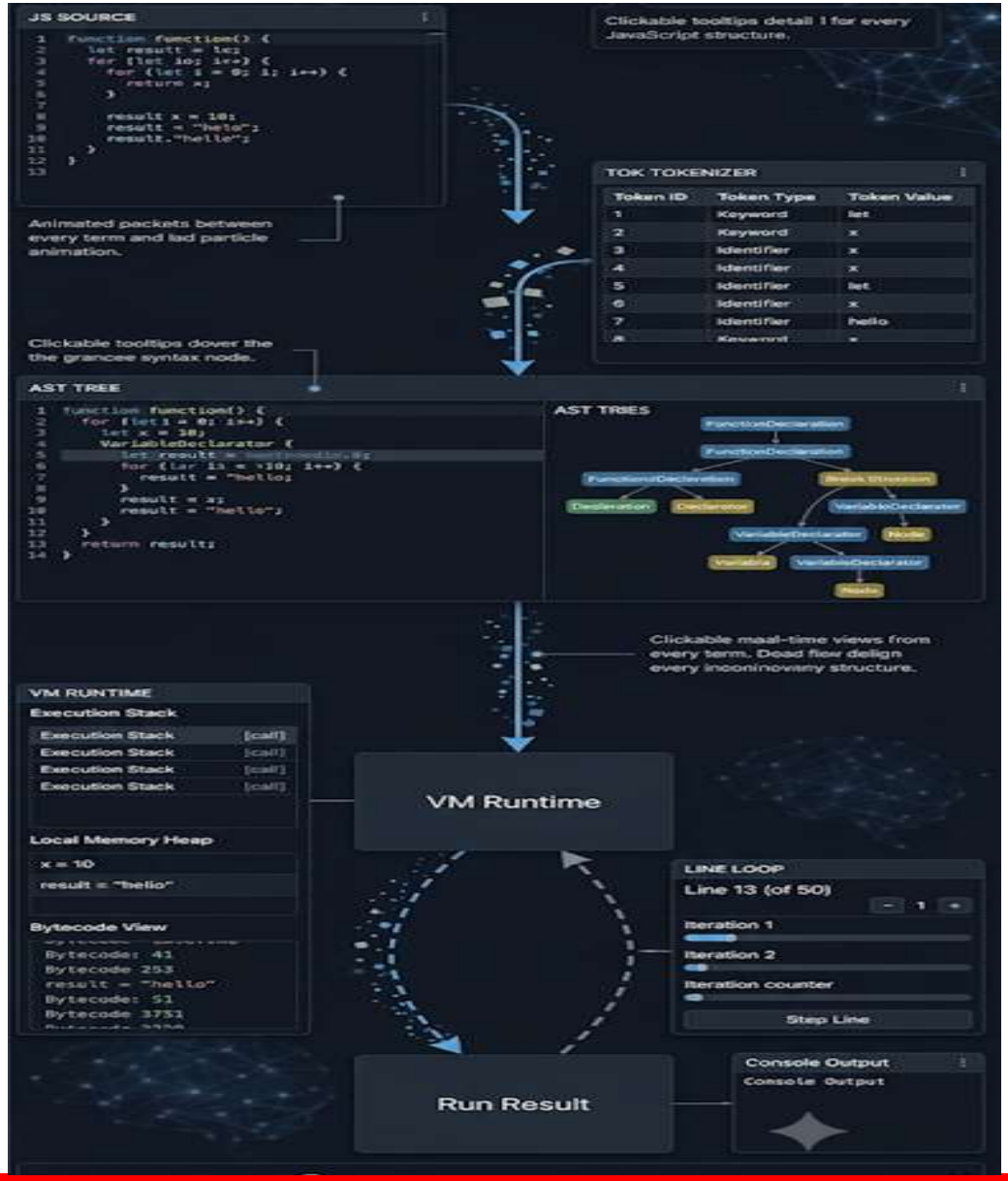
- **Advantages**
- Fast execution
- Detects many errors before execution
- Optimized machine code
- Suitable for large applications
- **Disadvantages**
- Compilation takes time
- Entire program must be error-free before execution

Interpreter

- An interpreter translates and executes the program **one statement at a time**.

Examples

- JavaScript
- Ruby
- BASIC



Interpreter Characteristics

CHITKARA UNIVERSITY

Advantages

- Easy debugging
- Immediate execution
- Suitable for learning

Disadvantages

- Slower execution
- No executable file generated

Compiler vs Interpreter

CHITKARA UNIVERSITY

Feature	Compiler	Interpreter
Translation	Entire program	Line by line
Speed	Faster	Slower
Error Reporting	After compilation	Immediate
Executable	Generated	Not generated
Debugging	Difficult	Easier

Sample Questions

CHITKARA UNIVERSITY

Which translator specifically converts Assembly language code into Machine Code?

1) Compiler

2) Interpreter

3) Assembler

4) Linker

sample Question (MCQS)

Which generation of languages relies on natural language processing or visual modeling (e.g., AI models, constraint-based systems)?

A) 2GL

B) 3GL

C) 4GL

D) 5GL

Sample Questions Scenerio based

Scenario: While compiling a C program, the compiler generates no syntax errors and creates `main.o`. However, when building the final executable, the process fails with the message `Undefined reference to 'calculate_interest()'`. *Which tool in the translation pipeline caused this failure?*

A) Assembler

B) Linker

C) Interpreter

D) Preprocessor

sample Questions Scenerio based

Scenario: An analyst runs a SQL query: `SELECT name, salary FROM employees WHERE department = 'Sales';` **without specifying how the computer should search the memory registers or iterate through disk blocks.** *This language belongs to which evolution stage of programming languages?*

A) First-Generation (1GL)

B) Second-Generation (2GL)

C) Third-Generation (3GL)

D) Fourth-Generation (4GL)

Solution

1. Assembler
2. 5GL
3. Linker
4. 4 GL

CHITKARA UNIVERSITY

Thank You